

Criptografia: Client-Side vs Server-Side

Dados em Repouso e em Trânsito na AWS

AWS Certified Developer Associate

Instrutor: Marlon Anesi

Dados em Repouso e em Trânsito

Encryption at Rest (em repouso)

- Dados armazenados (disco, banco, bucket)
- SSE: server-side encryption (AWS gerencia)
- CSE: client-side encryption (cliente gerencia)
- Objetivo: proteger se storage for comprometido
- Ex: S3 SSE-S3, SSE-KMS, EBS encrypted, RDS encrypted

Encryption in Transit (em trânsito)

- Dados em movimento pela rede
- TLS/SSL para HTTPS, VPN, TLS entre serviços
- Obrigatório para dados sensíveis
- Objetivo: proteger contra man-in-the-middle
- Ex: ACM (TLS cert), VPN, TLS endpoints AWS

Regra fundamental de segurança:

Criptografar dados SEMPRE: em repouso (comprometimento do storage) e em trânsito (comprometimento da rede). Complementares, não substitutos.

AWS oferece criptografia ativada por padrão em muitos serviços: S3, EBS, RDS, DynamoDB, Secrets Manager.

Server-Side Encryption (SSE): Modalidades no S3

PADRÃO

SSE-S3

Default (mais simples)

- AWS gerencia todo o ciclo de vida
- Algoritmo: AES-256
- Header: x-amz-server-side-encryption: AES256
- Sem custo adicional de API KMS
- Não é auditável no CloudTrail

AUDITORIA

SSE-KMS

Com controle via KMS

- Chave KMS (AWS Managed ou CMK)
- Header: x-amz-server-side-encryption: aws:kms
- Auditável no CloudTrail (cada acesso)
- Custo: chamadas KMS por operação
- Ideal para compliance e auditoria

CONTROLE TOTAL

SSE-C

Chave fornecida pelo cliente

- Cliente fornece chave em cada request
- AWS usa e descarta a chave
- Obrigatório HTTPS
- AWS não armazena a chave
- Controle total, mas gerenciamento complexo

CSE (Client-Side Encryption): cliente criptografa ANTES de enviar ao S3. AWS não tem acesso ao dado em nenhum momento.

Client-Side vs Server-Side: Quando Usar?

Aspecto	Server-Side (SSE)	Client-Side (CSE)
Quem criptografa	AWS (automatico na gravacao)	Aplicacao cliente (antes de enviar)
Chave gerenciada por	AWS KMS, S3, ou cliente (SSE-C)	Sempre pelo cliente (ex: KMS + SDK)
Dado trafegado na rede	Cleartext (se HTTPS, protegido em transito)	Ja criptografado antes do envio
AWS ve o dado?	SIM (no momento do decrypt/encrypt)	NAO (AWS nunca ve o plaintext)
Complexidade	Baixa (configuracao simples)	Alta (gerenciar keys e SDK)
Caso de uso	A maioria dos cenarios AWS	Requisitos de zero-trust ou compliance extremo

SSE em outros serviços AWS:

EBS	Encrypt at rest: chave KMS. Ativado na criação do volume.	DynamoDB	SSE ativado por padrão com AWS Owned Key ou CMK.
RDS	Encrypt at rest: KMS para instancia e snapshots.	Secrets Manager	Criptografa secrets com KMS (aws/secretsmanager ou CMK).

Boas Práticas e Pontos de Atenção

Cenário: Compliance (PCI, HIPAA)

- Use SSE-KMS com CMK para auditoria
- CloudTrail registra cada Encrypt/Decrypt
- Habilite S3 default encryption no bucket
- Force HTTPS: bucket policy com aws:SecureTransport

Cenário: Dados Ultra-Sensíveis

- Client-Side Encryption com AWS Encryption SDK
- AWS nunca ve o plaintext
- Gerenciar e fazer backup das chaves
- Ex: dados medicos, financeiros críticos

Pontos críticos para o exame:

- SSE-KMS: AWS ve o dado (gerencia chave). CSE: AWS nunca ve o dado. Esta diferenca cai no exame.
- SSE-C: HTTPS obrigatório. Cliente gerencia chave. AWS usa e descarta.
- S3 Bucket Policy pode forçar criptografia: Deny se x-amz-server-side-encryption ausente.
- aws:SecureTransport = false em Condition -> Deny: força HTTPS no bucket.

Pontos-chave para o Exame

1

SSE-KMS tem audit trail, SSE-S3 não

SSE-KMS: toda operação aparece no CloudTrail. SSE-S3: sem visibilidade individual. Para compliance, use SSE-KMS com CMK.

2

CSE: AWS nunca ve o plaintext

Dado já chega criptografado ao S3. Mesmo com acesso ao bucket, sem a chave não é possível ler. Zero-trust por design.

3

SSE-C: HTTPS obrigatório, sem armazenamento

Cliente envia chave no header HTTP. AWS usa para criptografar/descriptografar e descarta imediatamente. HTTPS obrigatório.

LEMBRE-SE: Precisa de auditoria de acesso? -> SSE-KMS! AWS não pode ver o dado? -> Client-Side Encryption!